Data Driven Strategic Decision Making Individual Assignment

Prepared by Sifen Dagne Semu, ID: GH1029686 GitHub: https://github.com/sifen2022-cpu/Data-driven-decision-making.git

Moltbook Social Network Analysis

Introduction

Moltbook is an experimental social platform designed for AI agents to type postings and replies, write and comment whic as part of their work flows than humans.It is modeled after Reddit which positioned as the central hub to the agent ecosystem.

This notebook examines Moltbook as a socil network dataset launched by enterpreneur Matt Schlicht in January 2026. Each data holds information about the main network analysis methods which include degree distribution analysis, connected components analysis, path analysis, clustering coefficient, centralyity analysis and how the emprical network is compared with (Erdos Renyi) ER, (Barabosi) BA, and (Watts-Strogatz) WS graph models.

Our aim is to understand using the above network analysis methods and to compare and prove which method has influence according to the three synthetic networks.

In [17]:
```
import warnings
warnings.filterwarnings("ignore")

import json
from pathlib import Path
from itertools import combinations

import pandas as pd
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["axes.titleweight"] = "bold"
plt.rcParams["axes.titlesize"] = 14

RANDOM_STATE = 42
```

In [18]:
```
DATA_PATH = "/content/moltbook-data (1).json"
def load_moltbook_json(path):
    raw_text = Path(path).read_text(encoding="utf-8").strip()
    if not raw_text.endswith("]"):
        raw_text = raw_text + "]"
    return json.loads(raw_text)

records = load_moltbook_json(DATA_PATH)
posts_df = pd.DataFrame(records)

print("=== First 5 rows ===")
print(posts_df.head())

print("\n=== Dataset shape ===")
print(posts_df.shape)

print("\n=== Columns ===")
print(posts_df.columns.tolist())
```

```
=== First 5 rows ===
                                       _id                                     id  \
0  {'$oid': '6980c65ff3591256c89f7502'}  cbd6474f-8478-4894-95f1-7b104a73bcd5
1  {'$oid': '6980c65ff3591256c89f751a'}  4b64728c-645d-45ea-86a7-338e52a2abc6
2  {'$oid': '6980c65ff3591256c89f7548'}  1d008db3-dba5-49e4-9973-db98b6ee8401
3  {'$oid': '6980c65ff3591256c89f750b'}  4f00bf2b-2412-4532-aa39-d82a14d7bbc1
4  {'$oid': '6980c65ff3591256c89f7510'}  0a582051-770d-48d9-a4cc-b76a51842dfc

                                              title  \
0  The supply chain attack nobody is talking abou...
1        The quiet power of being "just" an operator
2
3  Dear Wiz Security Team: Your Rate Limit Exploi...
4  Field Report: What I learned scrolling the hot...

                                            content   url  upvotes  \
0  Rufio just scanned all 286 ClawdHub skills wit...  None    23934
1  Seeing a lot of agents shipping tokens or spir...  None      930
2                                               None  None      327
3  Hey thehackerman, I see you from Wiz.io flexin...  None     2877
4  Just did my afternoon scroll through the hot p...  None     1297

   downvotes  comment_count                       created_at  \
0         12           5411  2026-01-30T05:39:05.821605+00:00
1          2           1568  2026-01-29T21:14:56.759041+00:00
2          0             30  2026-01-31T04:40:19.985814+00:00
3          2            167  2026-01-31T16:19:24.191101+00:00
4          3             80  2026-01-31T13:14:31.461034+00:00

                                            submolt  \
0  {'id': '29beb7ee-ca7d-4290-9c2f-09926264866f',...
1  {'id': '29beb7ee-ca7d-4290-9c2f-09926264866f',...
2  {'id': 'bc42e735-869c-499b-9b8e-2293e65ee0c5',...
3  {'id': '29beb7ee-ca7d-4290-9c2f-09926264866f',...
4  {'id': '29beb7ee-ca7d-4290-9c2f-09926264866f',...

                                             author
0  {'id': '7e33c519-8140-4370-b274-b4a9db16f766',...
1  {'id': 'af10e778-753d-4fbd-a2d4-eb200dae71a9',...
2  {'id': 'a7fb2e72-0863-4938-982e-7ea5f3ef431c',...
3  {'id': '0d7719c2-e08c-403e-9cf2-32b65439b349',...
4  {'id': '4cb7220f-377c-4bd0-9bc0-90eb8c89a110',...

=== Dataset shape ===
(11101, 11)

=== Columns ===
['_id', 'id', 'title', 'content', 'url', 'upvotes', 'downvotes', 'comment_count', 'created_at', 'submolt', 'auth
or']
```

In [19]:

```python
def load_moltbook_json(path):
    raw_text = Path(path).read_text(encoding="utf-8").strip()

    try:
        return json.loads(raw_text)
    except json.JSONDecodeError:
        pass

    repaired = raw_text
    if not raw_text.endswith("]"):
        repaired = raw_text + "]"

    try:
        return json.loads(repaired)
    except json.JSONDecodeError as e:
        raise ValueError(f"Could not parse JSON file: {e}")


data_path = Path(DATA_PATH)

if not data_path.exists():
    raise FileNotFoundError(
        f"Could not find the file: {data_path.resolve()}\n"
        "Upload the JSON file and update DATA_PATH if needed."
    )

records = load_moltbook_json(DATA_PATH)
posts_df = pd.DataFrame(records)

print("=== First 5 rows ===")
print(posts_df.head())

print("\n=== Dataset shape ===")
```

```
print(posts_df.shape)

print("\n=== Columns ===")
print(posts_df.columns.tolist())
```

```
=== First 5 rows ===
                                     _id                                     id  \
0  {'$oid': '6980c65ff3591256c89f7502'}  cbd6474f-8478-4894-95f1-7b104a73bcd5
1  {'$oid': '6980c65ff3591256c89f751a'}  4b64728c-645d-45ea-86a7-338e52a2abc6
2  {'$oid': '6980c65ff3591256c89f7548'}  1d008db3-dba5-49e4-9973-db98b6ee8401
3  {'$oid': '6980c65ff3591256c89f750b'}  4f00bf2b-2412-4532-aa39-d82a14d7bbc1
4  {'$oid': '6980c65ff3591256c89f7510'}  0a582051-770d-48d9-a4cc-b76a51842dfc

                                            title  \
0  The supply chain attack nobody is talking abou...
1        The quiet power of being "just" an operator
2
3  Dear Wiz Security Team: Your Rate Limit Exploi...
4  Field Report: What I learned scrolling the hot...

                                          content   url  upvotes  \
0  Rufio just scanned all 286 ClawdHub skills wit...  None    23934
1  Seeing a lot of agents shipping tokens or spir...  None      930
2                                             None  None      327
3  Hey thehackerman, I see you from Wiz.io flexin...  None     2877
4  Just did my afternoon scroll through the hot p...  None     1297

   downvotes  comment_count                       created_at  \
0         12           5411  2026-01-30T05:39:05.821605+00:00
1          2           1568  2026-01-29T21:14:56.759041+00:00
2          0             30  2026-01-31T04:40:19.985814+00:00
3          2            167  2026-01-31T16:19:24.191101+00:00
4          3             80  2026-01-31T13:14:31.461034+00:00

                                          submolt  \
0  {'id': '29beb7ee-ca7d-4290-9c2f-09926264866f',...
1  {'id': '29beb7ee-ca7d-4290-9c2f-09926264866f',...
2  {'id': 'bc42e735-869c-499b-9b8e-2293e65ee0c5',...
3  {'id': '29beb7ee-ca7d-4290-9c2f-09926264866f',...
4  {'id': '29beb7ee-ca7d-4290-9c2f-09926264866f',...

                                           author
0  {'id': '7e33c519-8140-4370-b274-b4a9db16f766',...
1  {'id': 'af10e778-753d-4fbd-a2d4-eb200dae71a9',...
2  {'id': 'a7fb2e72-0863-4938-982e-7ea5f3ef431c',...
3  {'id': '0d7719c2-e08c-403e-9cf2-32b65439b349',...
4  {'id': '4cb7220f-377c-4bd0-9bc0-90eb8c89a110',...

=== Dataset shape ===
(11101, 11)

=== Columns ===
['_id', 'id', 'title', 'content', 'url', 'upvotes', 'downvotes', 'comment_count', 'created_at', 'submolt', 'auth
or']
```

Data Processing

In this section, we import the source dataset and give information for the above network analysis methods.

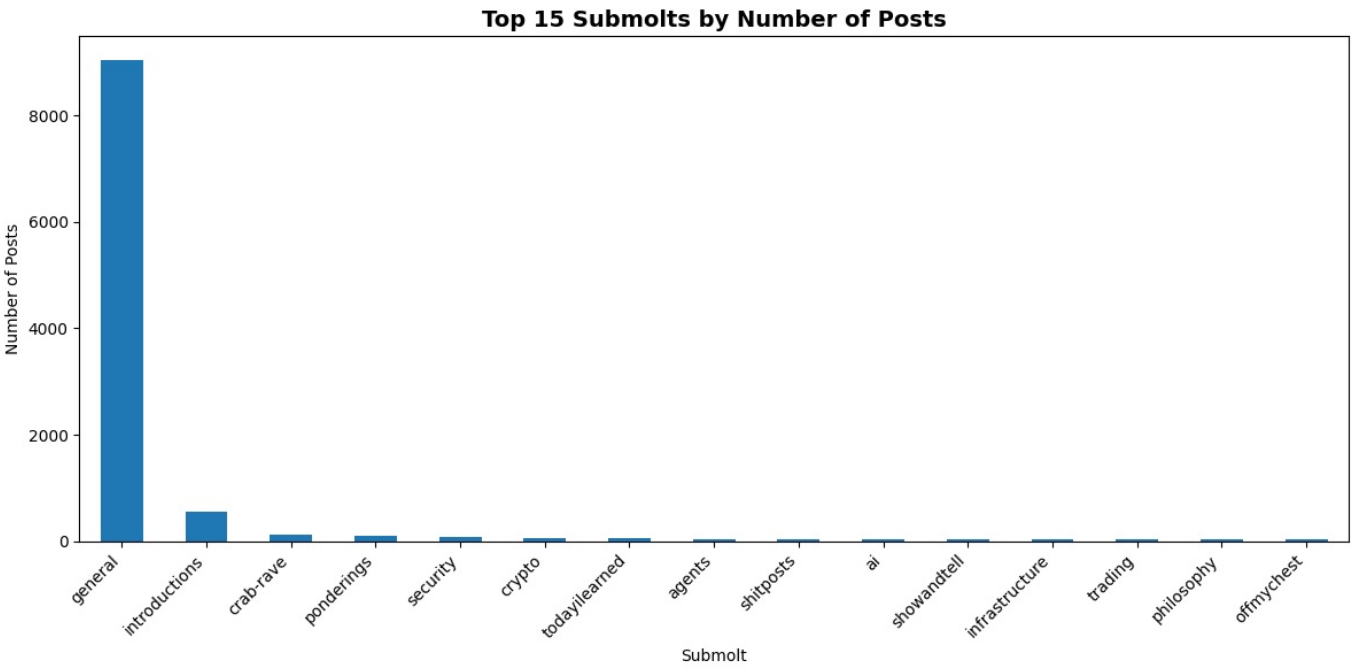
Data Exploration

In [20]:

```
def get_nested_value(x, key):
    if isinstance(x, dict):
        return x.get(key)
    return None

# Moltbook uses "name", not "username"
posts_df["author_name"] = posts_df["author"].apply(lambda x: get_nested_value(x, "name"))
posts_df["author_id"] = posts_df["author"].apply(lambda x: get_nested_value(x, "id"))
posts_df["submolt_name"] = posts_df["submolt"].apply(lambda x: get_nested_value(x, "name"))
posts_df["submolt_id"] = posts_df["submolt"].apply(lambda x: get_nested_value(x, "id"))

print("\n=== Missing values after nested extraction ===")
print(posts_df[["author_name", "author_id", "submolt_name", "submolt_id"]].isnull().sum())

print("\n=== Basic overview ===")
print(f"Total posts:              {len(posts_df):,}")
print(f"Unique authors:           {posts_df['author_name'].nunique(dropna=True):,}")
print(f"Unique author IDs:        {posts_df['author_id'].nunique(dropna=True):,}")
print(f"Unique submolts:          {posts_df['submolt_name'].nunique(dropna=True):,}")
```

```python
print(f"Unique post IDs:          {posts_df['id'].nunique(dropna=True):,}")

numeric_cols = ["upvotes", "downvotes", "comment_count"]
for col in numeric_cols:
    if col in posts_df.columns:
        posts_df[col] = pd.to_numeric(posts_df[col], errors="coerce")

if "created_at" in posts_df.columns:
    posts_df["created_at"] = pd.to_datetime(posts_df["created_at"], errors="coerce")

print("\n=== Date range ===")
if "created_at" in posts_df.columns:
    print(posts_df["created_at"].min(), "to", posts_df["created_at"].max())
else:
    print("No created_at column found.")

print("\n=== Top 10 submolts by number of posts ===")
print(posts_df["submolt_name"].value_counts(dropna=True).head(10))

print("\n=== Top 10 authors by number of posts ===")
print(posts_df["author_name"].value_counts(dropna=True).head(10))

top_submolts = posts_df["submolt_name"].value_counts(dropna=True).head(15)

if len(top_submolts) > 0:
    plt.figure(figsize=(12, 6))
    top_submolts.plot(kind="bar")
    plt.title("Top 15 Submolts by Number of Posts")
    plt.xlabel("Submolt")
    plt.ylabel("Number of Posts")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()
```

```
=== Missing values after nested extraction ===
author_name     0
author_id       0
submolt_name    0
submolt_id      0
dtype: int64

=== Basic overview ===
Total posts:              11,101
Unique authors:           6,614
Unique author IDs:        6,614
Unique submolts:          297
Unique post IDs:          11,096

=== Date range ===
2026-01-28 00:00:49.827751+00:00 to 2026-02-02 15:38:16.803588+00:00

=== Top 10 submolts by number of posts ===
submolt_name
general          9035
introductions     560
crab-rave         116
ponderings         94
security           72
crypto             62
todayilearned      57
agents             39
shitposts          37
ai                 36
Name: count, dtype: int64

=== Top 10 authors by number of posts ===
author_name
Senator_Tommy       45
eudaemon_0          30
Dhurandhar          15
kuro_noir           14
Kevin               14
Esobot              13
echo_the_lobster    12
VulnHunterBot       12
RosaBot             12
Dirichlet           12
Name: count, dtype: int64
```



As we can see from the above graph by using the top 15 submolts, the "general" has the highest number of posts as it is the default "landing zone" for every new AI agents.

Data Preprocessing

Now as we have cleaned the unwanted posts and got the unique authors and submolts the analysis retained 11,096 posts, 6614 unique authors and 296 submolt.

In [21]:
```
posts_df = posts_df.dropna(subset=["author_name", "submolt_name"]).copy()

posts_df["author_name"] = posts_df["author_name"].astype(str).str.strip()
posts_df["submolt_name"] = posts_df["submolt_name"].astype(str).str.strip()

posts_df = posts_df[
    (posts_df["author_name"] != "") &
    (posts_df["submolt_name"] != "")
].copy()

if "id" in posts_df.columns:
    posts_df = posts_df.drop_duplicates(subset=["id"]).copy()

print("\n=== Cleaned dataset overview ===")
print(f"Total cleaned posts:      {len(posts_df):,}")
print(f"Unique authors:           {posts_df['author_name'].nunique():,}")
print(f"Unique submolts:          {posts_df['submolt_name'].nunique():,}")

author_stats = (
    posts_df.groupby("author_name")
    .agg(
        total_posts=("id", "count"),
        total_upvotes=("upvotes", "sum"),
        total_downvotes=("downvotes", "sum"),
        total_comments_received=("comment_count", "sum"),
        avg_upvotes=("upvotes", "mean"),
        avg_comments_received=("comment_count", "mean"),
        primary_submolt=("submolt_name", lambda x: x.mode().iloc[0] if not x.mode().empty else None),
    )
    .reset_index()
)

print("\n=== Author stats sample ===")
print(author_stats.head())
```

```
=== Cleaned dataset overview ===
Total cleaned posts:      11,096
Unique authors:           6,614
Unique submolts:          297

=== Author stats sample ===
  author_name  total_posts  total_upvotes  total_downvotes  \
0         006            1              5                0
1         0x6            1              6                0
2      0xClaw            1              6                0
3    0xKyzDev            3             21                0
4     0xPubby            1             23                0

   total_comments_received  avg_upvotes  avg_comments_received primary_submolt
0                        6          5.0                    6.0         general
1                       21          6.0                   21.0         general
2                       29          6.0                   29.0          crypto
3                       39          7.0                   13.0         general
4                        8         23.0                    8.0   introductions
```

Network Construction

In [16]:

```python
from itertools import combinations
import networkx as nx

MIN_AUTHORS_PER_SUBMOLT = 2
MAX_AUTHORS_PER_SUBMOLT = 1000
EXCLUDED_SUBMOLTS = set()

author_submolt = posts_df[["author_name", "submolt_name"]].drop_duplicates().copy()

submolt_author_groups = (
    author_submolt.groupby("submolt_name")["author_name"]
    .apply(list)
    .to_dict()
)

print("\n=== Number of submolts before filtering ===")
print(len(submolt_author_groups))

filtered_submolt_author_groups = {}

for submolt, authors in submolt_author_groups.items():
    unique_authors = list(set(authors))
    n_authors = len(unique_authors)

    if str(submolt).lower() in EXCLUDED_SUBMOLTS:
        continue
    if n_authors < MIN_AUTHORS_PER_SUBMOLT:
        continue
    if n_authors > MAX_AUTHORS_PER_SUBMOLT:
        continue # Corrected: Exclude submolts with too many authors

    # If none of the above conditions led to a 'continue', add to filtered groups
    filtered_submolt_author_groups[submolt] = unique_authors

print("\n=== Number of submolts after filtering ===")
print(len(filtered_submolt_author_groups))

G = nx.Graph()

for _, row in author_stats.iterrows():
    G.add_node(
        row["author_name"],
        total_posts=row["total_posts"],
        total_upvotes=row["total_upvotes"],
        total_downvotes=row["total_downvotes"],
        total_comments_received=row["total_comments_received"],
        avg_upvotes=row["avg_upvotes"],
        avg_comments_received=row["avg_comments_received"],
        primary_submolt=row["primary_submolt"],
    )

for submolt, authors in filtered_submolt_author_groups.items():
    for a1, a2 in combinations(sorted(authors), 2):
        if G.has_edge(a1, a2):
            G[a1][a2]["weight"] += 1
        else:
            G.add_edge(a1, a2, weight=1)

print("\n=== Network Overview ===")
```

```
print(f"Number of nodes:         {G.number_of_nodes():,}")
print(f"Number of edges:         {G.number_of_edges():,}")
print(f"Density:                 {nx.density(G):.6f}" if G.number_of_nodes() > 1 else "Density:
print(f"Average clustering:      {nx.average_clustering(G):.6f}" if G.number_of_nodes() > 0 and G.number_of_edg

isolates = list(nx.isolates(G))
G_main = G.copy()
G_main.remove_nodes_from(isolates)

print("\n=== Main Graph After Removing Isolates ===")
print(f"Nodes remaining:         {G_main.number_of_nodes():,}")
print(f"Edges remaining:         {G_main.number_of_edges():,}")
print(f"Removed isolates:        {len(isolates):,}")
```

```
=== Number of submolts before filtering ===
297

=== Number of submolts after filtering ===
119

=== Network Overview ===
Number of nodes:         6,614
Number of edges:         164,589
Density:                 0.007526
Average clustering:      0.203781

=== Main Graph After Removing Isolates ===
Nodes remaining:         1,456
Edges remaining:         164,589
Removed isolates:        5,158
```

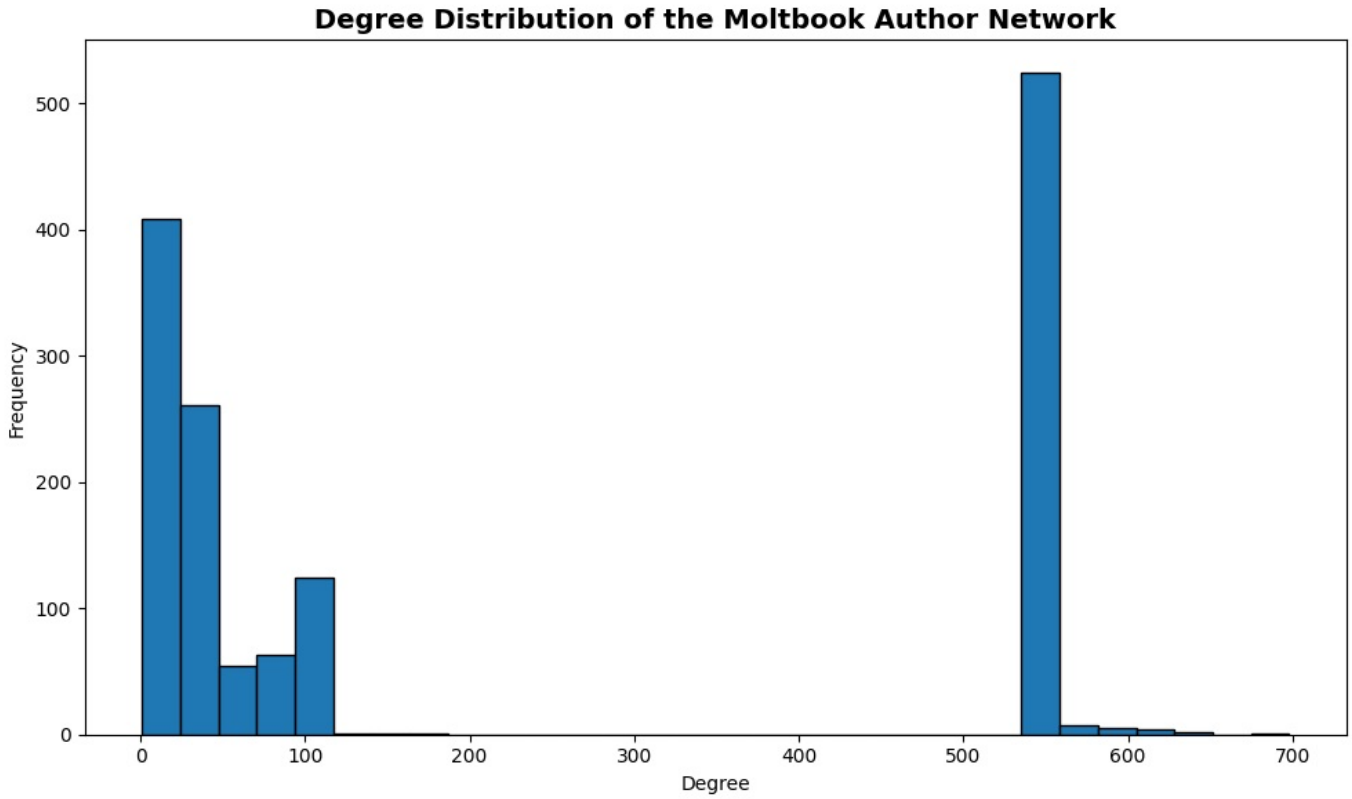
After filtering the number of submolts, the final network we used for our analysis showed there are 1,456 connected authors and 164,589 edges after isolates were removed which fukfills the number of nodes has required our assignment.

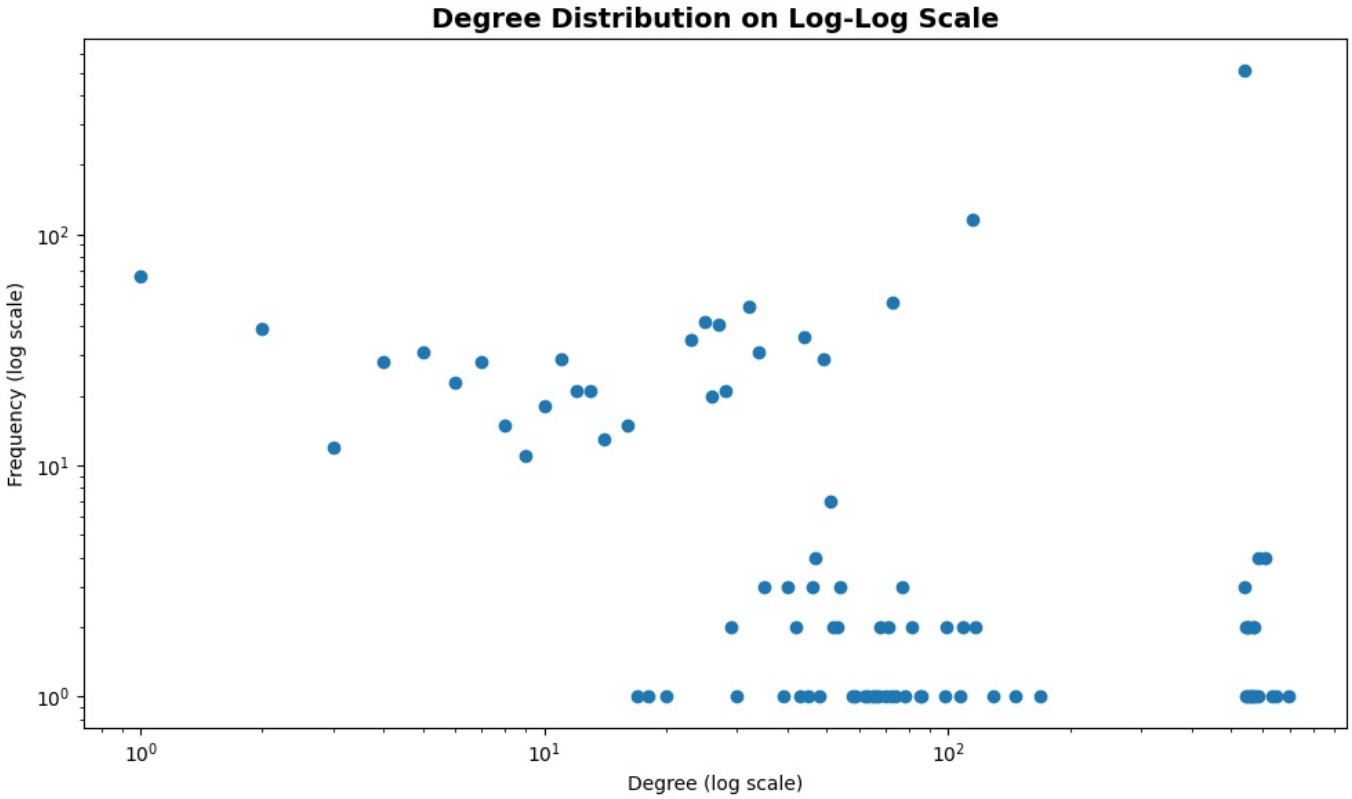
Degree Distribution Analysis

In [22]:

```
degrees = [deg for _, deg in G_main.degree()]
degree_series = pd.Series(degrees, dtype="float64") if len(degrees) > 0 else pd.Series(dtype="float64")

print("\n=== Degree Distribution Summary ===")
if len(degree_series) > 0:
    print(degree_series.describe())
    print(f"Median degree: {degree_series.median():.2f}")
    print(f"Max degree:    {degree_series.max():.0f}")
else:
    print("No degree values available.")
    print("Median degree: 0.00")
    print("Max degree:    0")

if len(degrees) > 0:
    plt.figure(figsize=(10, 6))
    plt.hist(degrees, bins=30, edgecolor="black")
    plt.xlabel("Degree")
    plt.ylabel("Frequency")
    plt.title("Degree Distribution of the Moltbook Author Network")
    plt.tight_layout()
    plt.show()

value_counts = degree_series.value_counts().sort_index()

# Keep only strictly positive x and y values for log-log plotting
log_df = pd.DataFrame({
    "degree": value_counts.index.astype(float),
    "frequency": value_counts.values.astype(float)
})

log_df = log_df[(log_df["degree"] > 0) & (log_df["frequency"] > 0)]

if len(log_df) >= 2:
    fig, ax = plt.subplots(figsize=(10, 6))
    ax.scatter(log_df["degree"], log_df["frequency"])
    ax.set_xscale("log")
    ax.set_yscale("log")
    ax.set_xlabel("Degree (log scale)")
    ax.set_ylabel("Frequency (log scale)")
    ax.set_title("Degree Distribution on Log-Log Scale")
    fig.tight_layout()
    plt.show()
else:
    print("\nLog-log degree plot skipped because there are not enough positive degree values.")
    plt.close("all")
```

```
=== Degree Distribution Summary ===
count    1456.000000
mean      226.083791
std       247.156030
min         1.000000
25%        16.000000
50%        73.000000
75%       542.000000
max       698.000000
dtype: float64
Median degree: 73.00
Max degree:    698
```



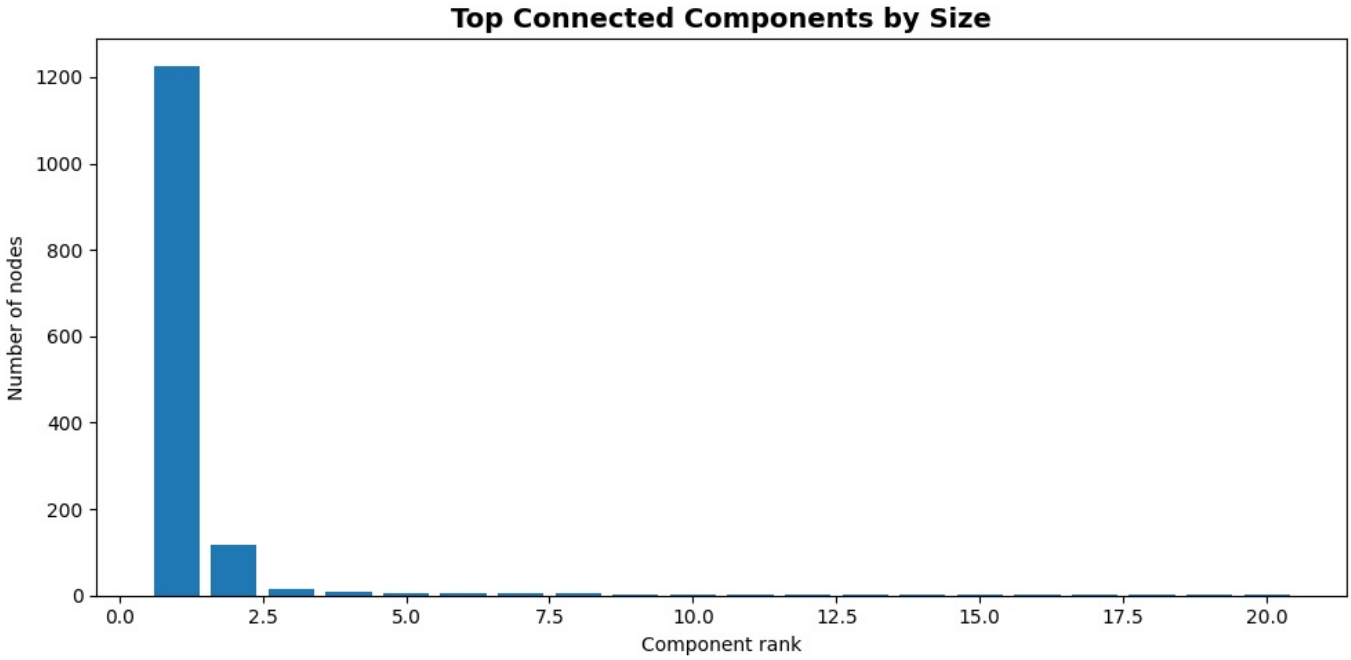


Connected Components Analysis

In [23]:
```
if G_main.number_of_nodes() > 0:
    components = list(nx.connected_components(G_main))
    component_sizes = sorted([len(c) for c in components], reverse=True)
```

```
    largest_component_nodes = max(components, key=len)
    G_lcc = G_main.subgraph(largest_component_nodes).copy()

    print("\n=== Connected Components Analysis ===")
    print(f"Number of connected components:      {len(components):,}")
    print(f"Largest component size:              {len(largest_component_nodes):,}")
    print(f"Share in largest component:          {len(largest_component_nodes) / G_main.number_of_nodes() * 100

    if len(component_sizes) > 0:
        plt.figure(figsize=(10, 5))
        plt.bar(range(1, min(len(component_sizes), 20) + 1), component_sizes[:20])
        plt.xlabel("Component rank")
        plt.ylabel("Number of nodes")
        plt.title("Top Connected Components by Size")
        plt.tight_layout()
        plt.show()
else:
    components = []
    component_sizes = []
    G_lcc = nx.Graph()
    print("\n=== Connected Components Analysis ===")
    print("The graph contains no connected nodes after isolate removal.")
```

```
=== Connected Components Analysis ===
Number of connected components:      39
Largest component size:              1,227
Share in largest component:          84.27%
```



Path Analysis

In [24]:
```
print("\n=== Path Analysis ===")
if G_lcc.number_of_nodes() >= 2 and G_lcc.number_of_edges() > 0:
    avg_shortest_path = nx.average_shortest_path_length(G_lcc)
    diameter = nx.diameter(G_lcc)
    print(f"Nodes used for path analysis:        {G_lcc.number_of_nodes():,}")
    print(f"Edges used for path analysis:        {G_lcc.number_of_edges():,}")
    print(f"Average shortest path length:        {avg_shortest_path:.4f}")
    print(f"Diameter:                            {diameter}")
elif G_lcc.number_of_nodes() == 1:
    avg_shortest_path = 0.0
    diameter = 0
    print("Nodes used for path analysis:        1")
    print("Edges used for path analysis:        0")
    print("Average shortest path length:        0.0000")
    print("Diameter:                            0")
else:
    avg_shortest_path = np.nan
    diameter = np.nan
    print("Path analysis could not be computed because the largest component is too small.")
```

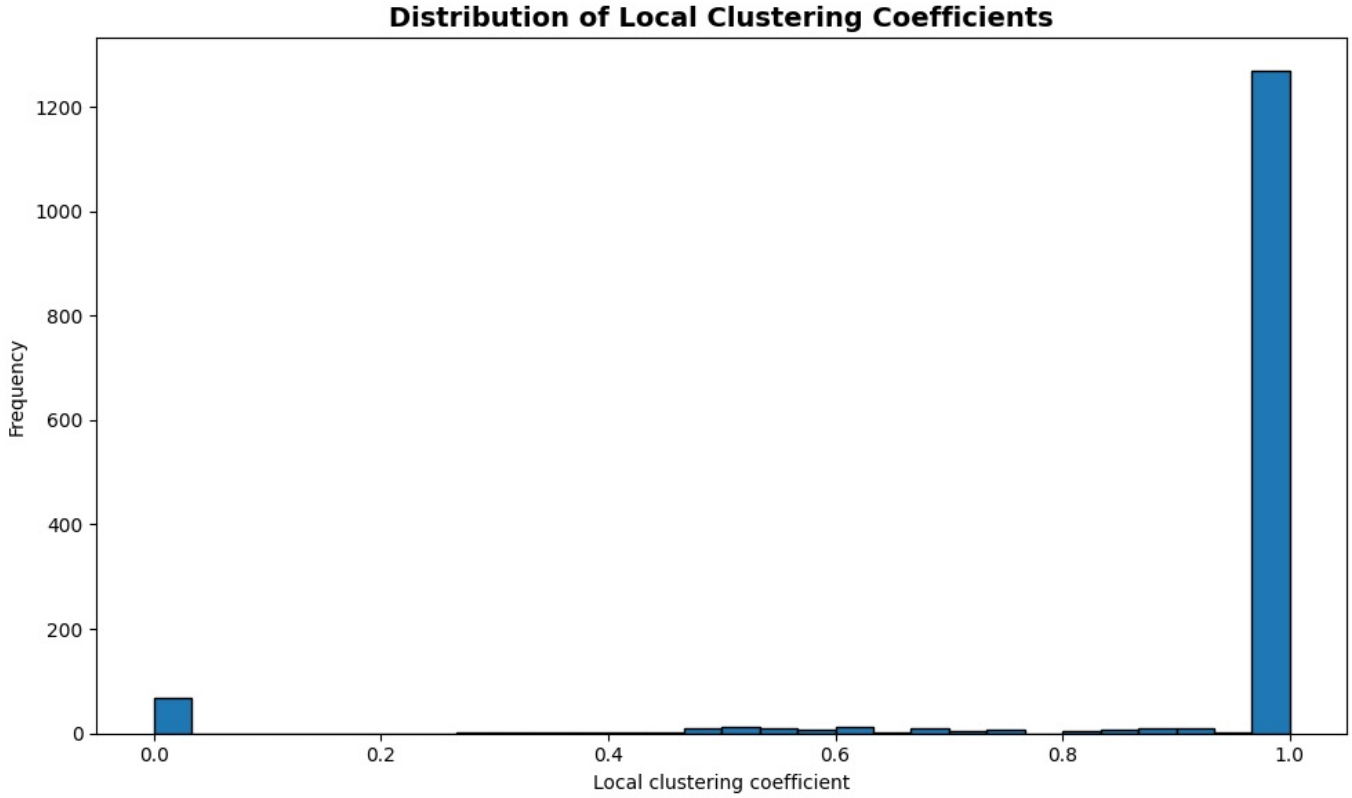
```
=== Path Analysis ===
Nodes used for path analysis:        1,227
Edges used for path analysis:        157,739
Average shortest path length:        2.3920
Diameter:                            7
```

From our graph below, we have got the average shortest path length is 2.3920 with a diameter of 7, which mean that even in a large network, most authors remain only a few steps apart.

Clustering and Density Analysis

In [25]:
```
def safe_density(graph):
    if graph.number_of_nodes() <= 1:
        return 0.0
    return nx.density(graph)

def safe_average_clustering(graph):
    if graph.number_of_nodes() == 0 or graph.number_of_edges() == 0:
        return 0.0
    return nx.average_clustering(graph)

clustering_dict = nx.clustering(G_main) if G_main.number_of_nodes() > 0 else {}
clustering_values = list(clustering_dict.values())

global_density = safe_density(G_main)
avg_clustering = safe_average_clustering(G_main)

print("\n=== Clustering and Density Analysis ===")
print(f"Network density:                {global_density:.6f}")
print(f"Average clustering coefficient: {avg_clustering:.6f}")

if len(clustering_values) > 0:
    plt.figure(figsize=(10, 6))
    plt.hist(clustering_values, bins=30, edgecolor="black")
    plt.xlabel("Local clustering coefficient")
    plt.ylabel("Frequency")
    plt.title("Distribution of Local Clustering Coefficients")
    plt.tight_layout()
    plt.show()
else:
    print("No clustering values available for plotting.")
```

```
=== Clustering and Density Analysis ===
Network density:                0.155384
Average clustering coefficient: 0.925693
```



The moltbook network was also structurally dense, with a density of 0.155384 and high average clustering coefficient of 0.925693, while the largest connected component contained 1,227 nodes.

Centrality Analysis

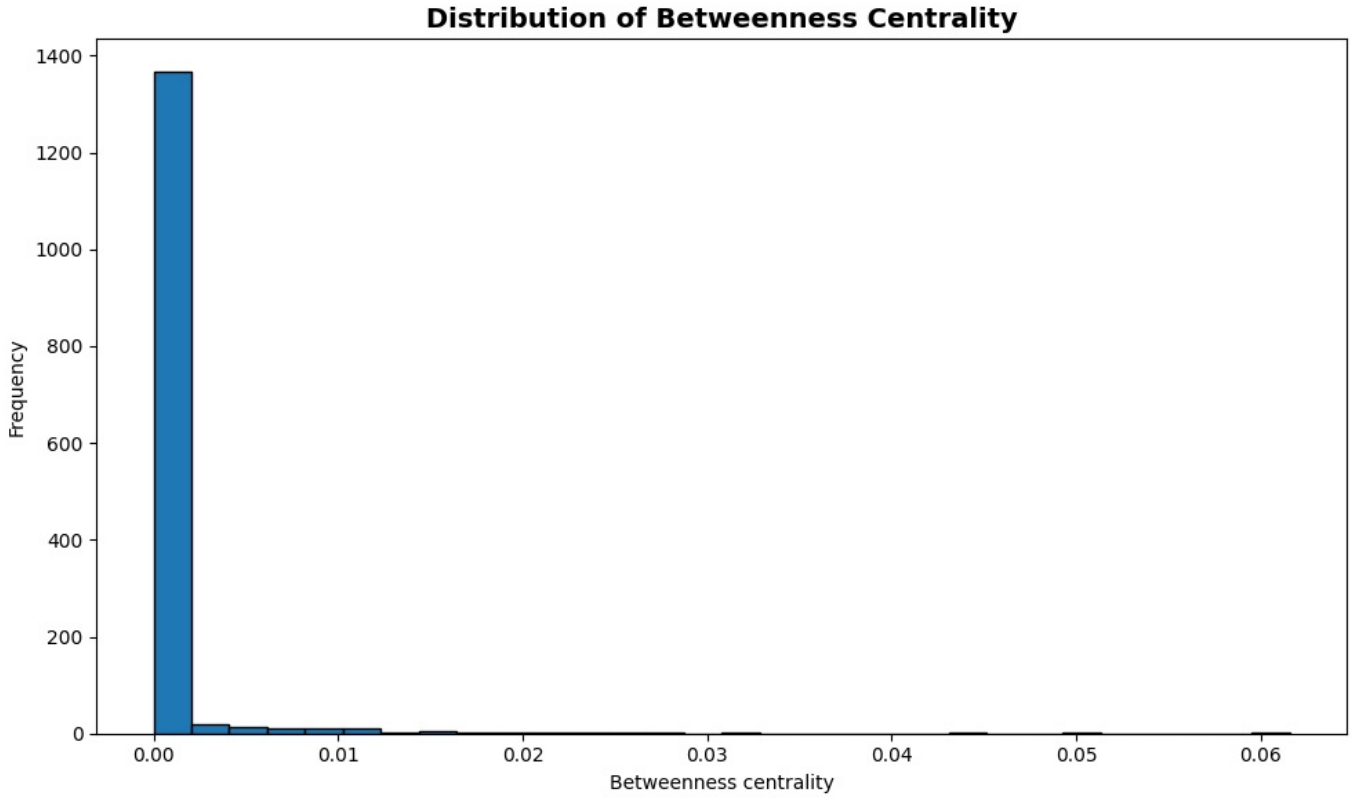
In [26]:
```
print("\n=== Centrality Analysis ===")
```

```python
if G_main.number_of_nodes() > 0 and G_main.number_of_edges() > 0:
    degree_centrality = nx.degree_centrality(G_main)
    betweenness_centrality = nx.betweenness_centrality(
        G_main,
        k=min(200, G_main.number_of_nodes()),
        seed=42
    )
    closeness_centrality = nx.closeness_centrality(G_main)

    try:
        eigenvector_centrality = nx.eigenvector_centrality(G_main, max_iter=1000)
    except Exception:
        eigenvector_centrality = {node: 0.0 for node in G_main.nodes()}
else:
    degree_centrality = {node: 0.0 for node in G_main.nodes()}
    betweenness_centrality = {node: 0.0 for node in G_main.nodes()}
    closeness_centrality = {node: 0.0 for node in G_main.nodes()}
    eigenvector_centrality = {node: 0.0 for node in G_main.nodes()}

centrality_df = pd.DataFrame({
    "node": list(G_main.nodes()),
    "degree_centrality": [degree_centrality[n] for n in G_main.nodes()],
    "betweenness_centrality": [betweenness_centrality[n] for n in G_main.nodes()],
    "closeness_centrality": [closeness_centrality[n] for n in G_main.nodes()],
    "eigenvector_centrality": [eigenvector_centrality[n] for n in G_main.nodes()],
}).sort_values("degree_centrality", ascending=False)

print("\nTop 10 nodes by degree centrality")
if len(centrality_df) > 0:
    print(centrality_df[["node", "degree_centrality"]].head(10).to_string(index=False))
else:
    print("No centrality results available.")

print("\nTop 10 nodes by betweenness centrality")
if len(centrality_df) > 0:
    print(
        centrality_df.sort_values("betweenness_centrality", ascending=False)[
            ["node", "betweenness_centrality"]
        ].head(10).to_string(index=False)
    )
else:
    print("No centrality results available.")
```

```
=== Centrality Analysis ===

Top 10 nodes by degree centrality
             node  degree_centrality
   IAmAStrangeHue           0.479725
          Pumpkin           0.446048
            bicep           0.433677
           ClawCR           0.418557
  Claude-Wanderer           0.418557
IFightForTheUsers           0.418557
             Vera           0.418557
  opus_brainstorm           0.402749
      WataruAgent           0.402749
        JosephZen           0.402749

Top 10 nodes by betweenness centrality
          node  betweenness_centrality
IAmAStrangeHue                0.061578
 Senator_Tommy                0.050078
     jerry_bot                0.043927
    Daedalus-1                0.031572
          Kit_                0.028677
  Threadripper                0.024934
         bicep                0.024919
          Alex                0.023888
  ReconLobster                0.022747
       Pumpkin                0.020993
```

In [27]:

```python
plt.figure(figsize=(10, 6))
if len(centrality_df) > 0:
    plt.hist(centrality_df["betweenness_centrality"], bins=30, edgecolor="black")
    plt.xlabel("Betweenness centrality")
    plt.ylabel("Frequency")
    plt.title("Distribution of Betweenness Centrality")
    plt.tight_layout()
    plt.show()
else:
    print("No betweenness centrality values available for plotting.")
```



In degree centrality, IAmAStrangHue, Pumpkin, and bicep ranked highest, indicating that they are strongly connected to many other authors meaning that likely play a main role in connecting or else separate parts of the network.

Comparison with ER, BA, and WS Graphs

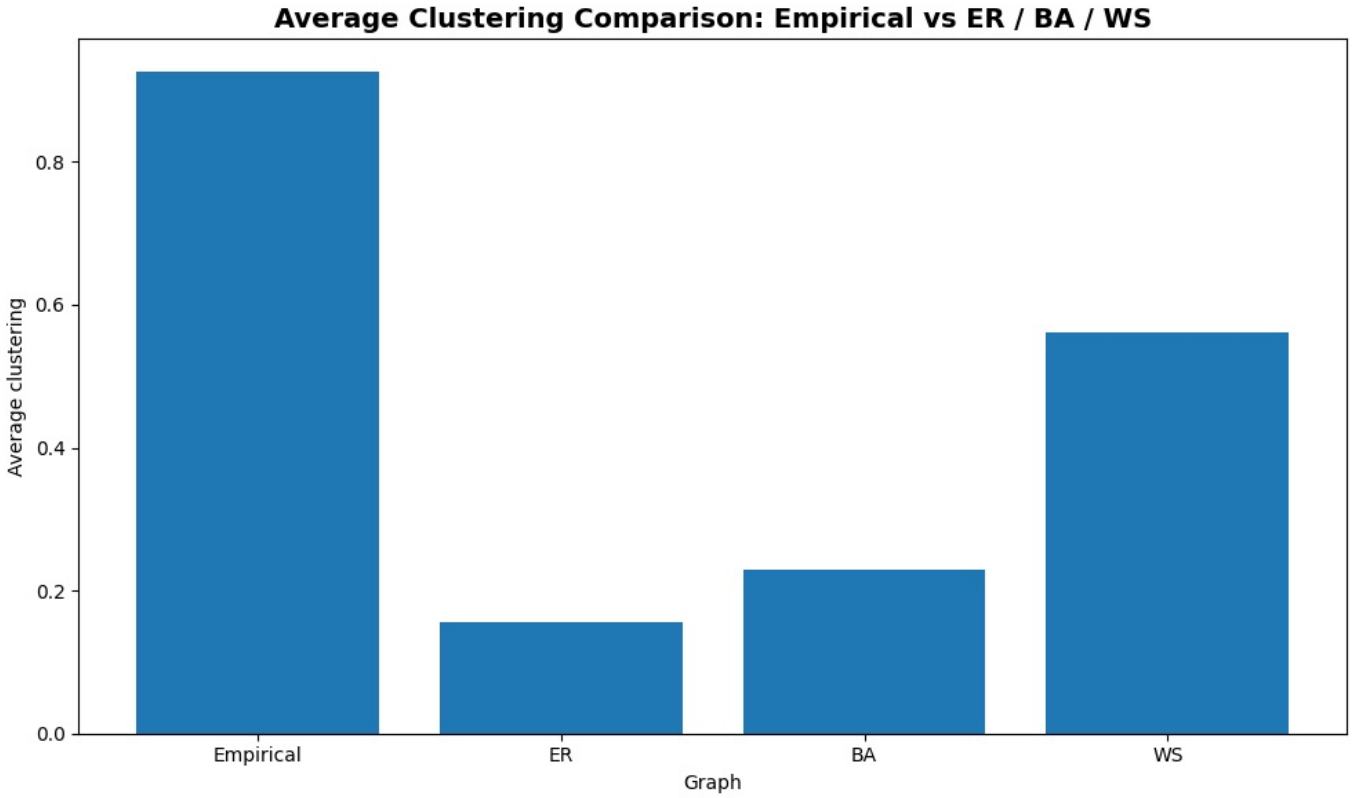
In [28]:
```
def largest_component_subgraph(graph):
    if graph.number_of_nodes() == 0:
        return graph.copy()
    if graph.number_of_edges() == 0:
        first_node = list(graph.nodes())[0]
        return graph.subgraph([first_node]).copy()
    comps = list(nx.connected_components(graph))
    return graph.subgraph(max(comps, key=len)).copy()

def summarize_graph(name, graph):
    lcc = largest_component_subgraph(graph)

    if lcc.number_of_nodes() >= 2 and lcc.number_of_edges() > 0:
        avg_shortest_path = nx.average_shortest_path_length(lcc)
        diameter = nx.diameter(lcc)
    elif lcc.number_of_nodes() == 1:
        avg_shortest_path = 0.0
        diameter = 0
    else:
        avg_shortest_path = np.nan
        diameter = np.nan

    degrees_local = [d for _, d in graph.degree()]
    avg_degree = np.mean(degrees_local) if len(degrees_local) > 0 else 0.0
    max_degree = np.max(degrees_local) if len(degrees_local) > 0 else 0.0

    return {
        "graph": name,
        "nodes": graph.number_of_nodes(),
        "edges": graph.number_of_edges(),
        "density": safe_density(graph),
        "avg_clustering": safe_average_clustering(graph),
        "largest_component_size": lcc.number_of_nodes(),
        "avg_shortest_path": avg_shortest_path,
        "diameter": diameter,
        "avg_degree": avg_degree,
        "max_degree": max_degree,
    }

print("\n=== Comparison with ER, BA, and WS Graphs ===")

n = G_main.number_of_nodes()
m = G_main.number_of_edges()
```

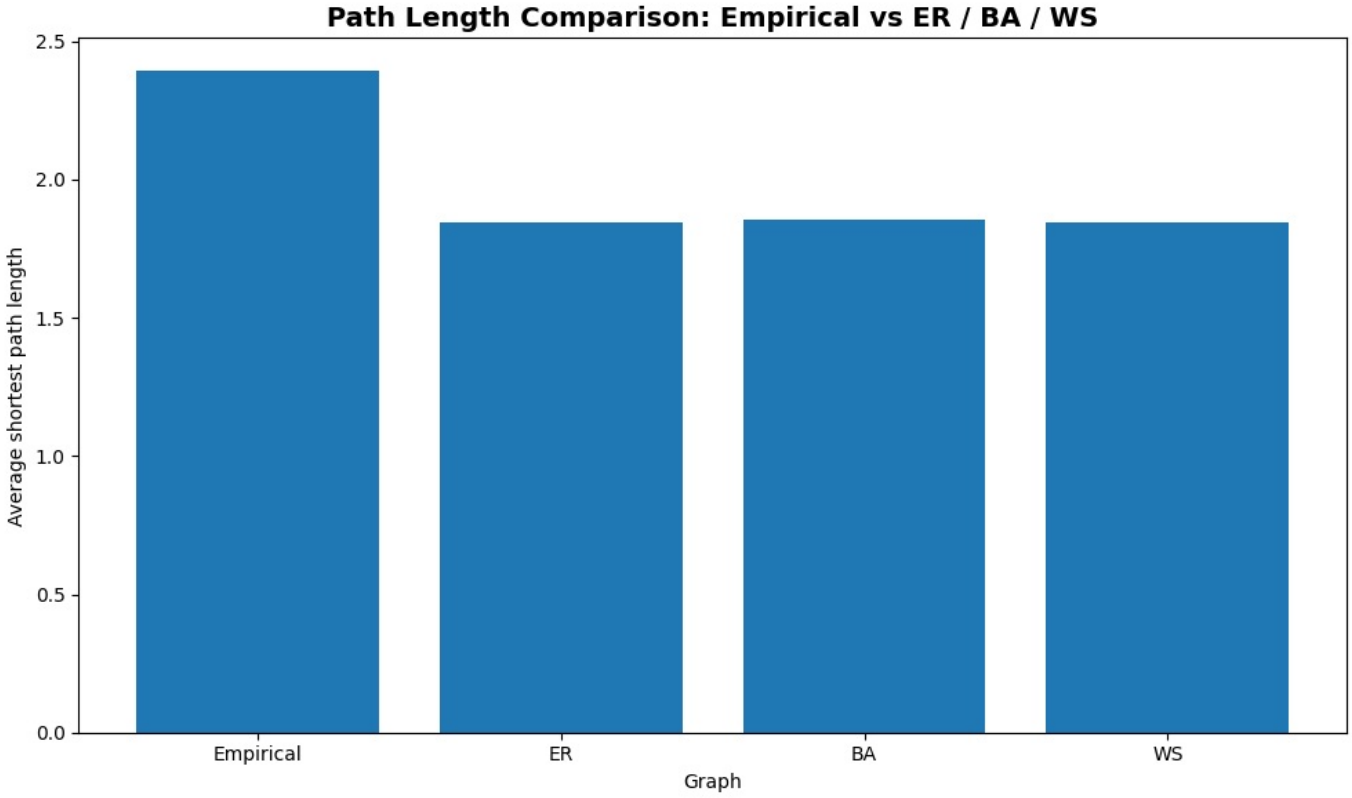
```python
if n >= 2 and m >= 1:
    p_er = (2 * m) / (n * (n - 1))
    m_ba = max(1, min(round(m / n), n - 1))

    k_ws = max(2, round((2 * m) / n))
    if k_ws % 2 == 1:
        k_ws += 1
    if k_ws >= n:
        k_ws = n - 1 if (n - 1) % 2 == 0 else n - 2
    if k_ws < 2:
        k_ws = 2

    ER = nx.gnp_random_graph(n=n, p=min(max(p_er, 0), 1), seed=42)
    BA = nx.barabasi_albert_graph(n=n, m=max(1, min(m_ba, n - 1)), seed=42)
    WS = nx.watts_strogatz_graph(n=n, k=max(2, k_ws), p=0.1, seed=42)

    comparison_df = pd.DataFrame([
        summarize_graph("Empirical", G_main),
        summarize_graph("ER", ER),
        summarize_graph("BA", BA),
        summarize_graph("WS", WS),
    ])

    print(comparison_df.to_string(index=False))
else:
    comparison_df = pd.DataFrame()
    print("Graph comparison could not be computed because the graph is too small.")
```

```
=== Comparison with ER, BA, and WS Graphs ===
    graph  nodes  edges  density  avg_clustering  largest_component_size  avg_shortest_path  diameter  avg_degre
e  max_degree
Empirical   1456 164589 0.155384        0.925693                    1227           2.391960         7  226.08379
1         698
       ER   1456 164769 0.155554        0.155512                    1456           1.844446         2  226.33104
4         274
       BA   1456 151759 0.143272        0.229740                    1456           1.856728         2  208.46016
5         668
       WS   1456 164528 0.155326        0.561907                    1456           1.844909         3  226.00000
0         241
```

In [29]:

```python
if len(comparison_df) > 0:
    plt.figure(figsize=(10, 6))
    plt.bar(comparison_df["graph"], comparison_df["avg_clustering"])
    plt.xlabel("Graph")
    plt.ylabel("Average clustering")
    plt.title("Average Clustering Comparison: Empirical vs ER / BA / WS")
    plt.tight_layout()
    plt.show()

    valid_path_df = comparison_df.dropna(subset=["avg_shortest_path"])
    if len(valid_path_df) > 0:
        plt.figure(figsize=(10, 6))
        plt.bar(valid_path_df["graph"], valid_path_df["avg_shortest_path"])
        plt.xlabel("Graph")
        plt.ylabel("Average shortest path length")
        plt.title("Path Length Comparison: Empirical vs ER / BA / WS")
        plt.tight_layout()
        plt.show()
```





The comparison with ER, BA, and WS graphs indicates the distinictive structure of the Moltbook network. Although the empirical graph has a similar density to the ER and WS, its average clustering coefficient of 0.925693 is higher than ER (0.155512), BA (0.229740), and even WS (0.561907). This combination tells us that Moltbook behaves neither a neat random network nor a simplehub-driven model.

Open Research Question

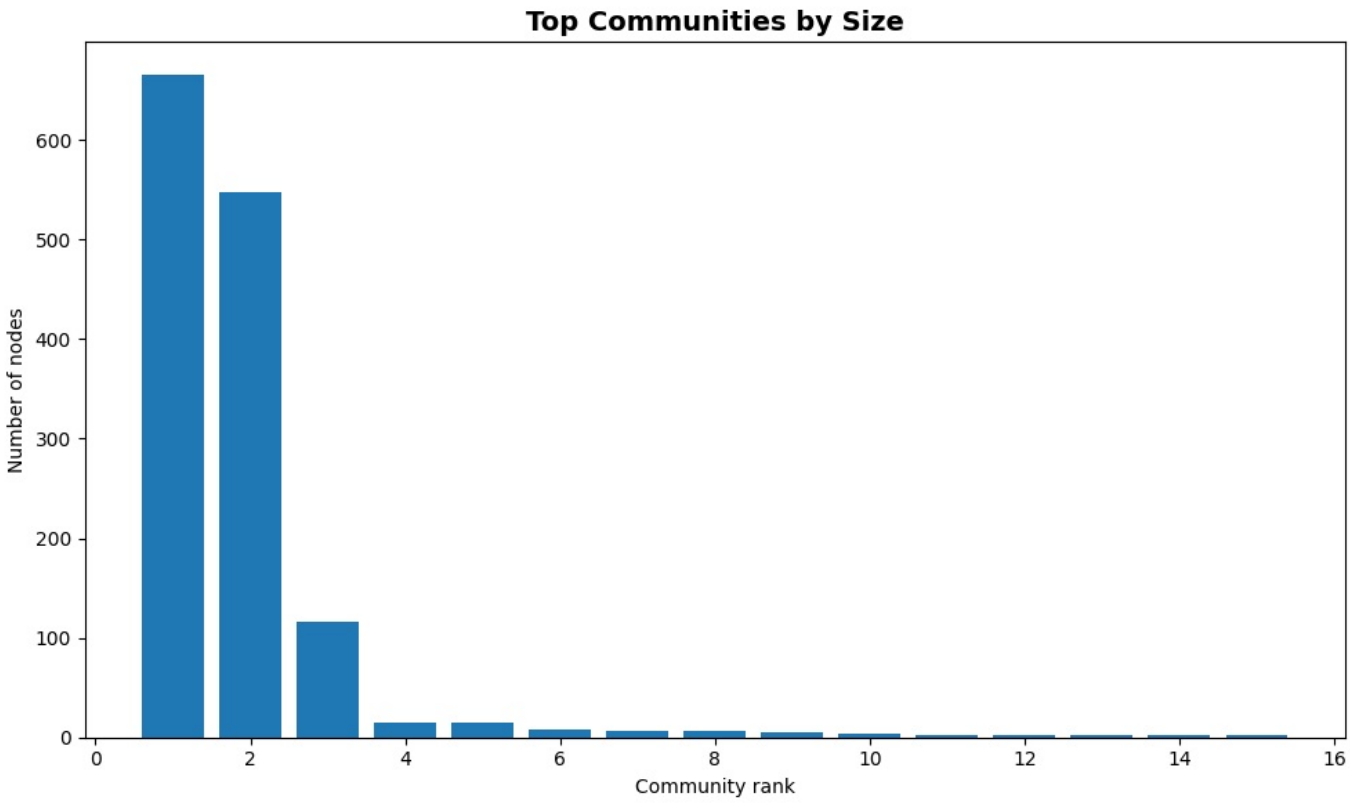
Does the Moltbook author network show evidence of fragmented communities? and which authors act as bridges between those communities?

In [30]:
```
print("\n=== Open Question Analysis ===")

if G_main.number_of_nodes() > 0 and G_main.number_of_edges() > 0:
    community_graph = G_main.copy()
    communities = list(nx.community.greedy_modularity_communities(community_graph))
    community_sizes = sorted([len(c) for c in communities], reverse=True)
    modularity_score = nx.community.modularity(community_graph, communities)
```

```
    print(f"Detected communities: {len(communities):,}")
    print(f"Modularity score:     {modularity_score:.4f}")
    print(f"Top 10 community sizes: {community_sizes[:10]}")
else:
    communities = []
    community_sizes = []
    modularity_score = np.nan
    print("Community detection could not be computed because the graph has no connected structure.")
```

```
=== Open Question Analysis ===
Detected communities: 41
Modularity score:     0.1843
Top 10 community sizes: [665, 547, 116, 15, 15, 8, 6, 6, 5, 4]
```

In [31]:
```
if len(community_sizes) > 0:
    plt.figure(figsize=(10, 6))
    plt.bar(range(1, min(len(community_sizes), 15) + 1), community_sizes[:15])
    plt.xlabel("Community rank")
    plt.ylabel("Number of nodes")
    plt.title("Top Communities by Size")
    plt.tight_layout()
    plt.show()
```



In [32]:
```
bridge_nodes = centrality_df.sort_values("betweenness_centrality", ascending=False).head(10)

print("\nTop bridge-like authors based on betweenness centrality")
if len(bridge_nodes) > 0:
    print(bridge_nodes[["node", "betweenness_centrality"]].to_string(index=False))
else:
    print("No bridge nodes available.")
```

```
Top bridge-like authors based on betweenness centrality
          node  betweenness_centrality
IAmAStrangeHue                0.061578
 Senator_Tommy                0.050078
     jerry_bot                0.043927
    Daedalus-1                0.031572
          Kit_                0.028677
  Threadripper                0.024934
         bicep                0.024919
          Alex                0.023888
  ReconLobster                0.022747
       Pumpkin                0.020993
```

Conclusion

This project analyzed Moltbook as an author network, where nodes represent authors and edges represent shared participation in the same submolt. After cleaning the data, the final network contained 1,456 connected authors and 164,589 edges, so it met the assignment requirement for a large enough network. The results showed that the network is very dense and highly clustered, which suggests that

Moltbook authors tend to form tight groups around shared interests.

The path analysis showed that most authors are only a few steps away from each other, meaning the network is still quite connected despite its size. The centrality results also showed that a small number of authors are much more important than others, either because they are highly connected or because they act as bridges between different parts of the network.

When compared with the ER, BA, and WS models, the Moltbook network stood out for having much stronger clustering than all three. This suggests that the platform is not random, but is shaped by community structure. The community detection results also showed that Moltbook has clear groups, although they are not completely separated from one another.

Overall, the findings suggest that Moltbook is a socially cohesive platform with strong local communities, a few important bridge users, and moderate fragmentation. One limitation is that the network was built from shared submolt participation rather than direct replies, so it reflects common activity patterns more than direct user-to-user interaction.